

```

import { NextRequest, NextResponse } from "next/server";
import { z } from "zod";
import { requireUser } from "@lib/auth";
import { db } from "@lib/db";
import { NodeSchema, type SchemaResult } from "@lib/ai/schema-generator";

const BodySchema = z.object({
  ideaId: z.string(),
  node: NodeSchema,
});

export async function POST(req: NextRequest) {
  let user;
  try {
    user = await requireUser();
  } catch {
    return NextResponse.json({ error: "Non connecté." }, { status: 401 });
  }

  const json = await req.json().catch(() => null);
  const parsed = BodySchema.safeParse(json);
  if (!parsed.success) {
    return NextResponse.json({ error: "Requête invalide." }, { status: 400 });
  }
  const { ideaId, node } = parsed.data;

  const idea = await db.idea.findFirst({ where: { id: ideaId, userId: user.id } });
  if (!idea) {
    return NextResponse.json({ error: "Projet introuvable." }, { status: 404 });
  }

  const currentSchema = JSON.parse(idea.schemaData) as SchemaResult;
  const updatedSchema: SchemaResult = { ...currentSchema, nodes: [...currentSchema.nodes, node] };

  await db.idea.update({
    where: { id: idea.id },
    data: { schemaData: JSON.stringify(updatedSchema) },
  });
  await db.historyEntry.create({
    data: { ideaId: idea.id, type: "schema_update", summary: `Ajout au schéma : "${node.summary}"` },
  });

  // Cascade demandée par Raphaël : si le schéma change, l'horizon des
  // quêtes n'est plus forcément pertinent (les quêtes ont été générées sur

```

```
// la base de l'ancien schéma). On supprime les quêtes PENDING des deux
// pistes – pas les DONE, on garde l'historique – pour qu'elles se
// régénèrent au prochain chargement, avec les garde-fous anti-boucle déjà
// en place (lib/quests.ts : pas plus d'une génération payante par piste
// toutes les 20 secondes, et repli sur une quête de secours en cas
// d'échec IA).
await db.quest.deleteMany({ where: { ideaId: idea.id, status: "PENDING" } });

return NextResponse.json({ schema: updatedSchema });
}
```